

The Use Of Genetic Algorithms In Airfoil Design And CFD

Aim

To explore the use of genetic algorithms to speed up airfoil design, and how they can help the most. I tested 3 different optimization methods, each altering different factors, and recorded the progress through generations.

Definitions

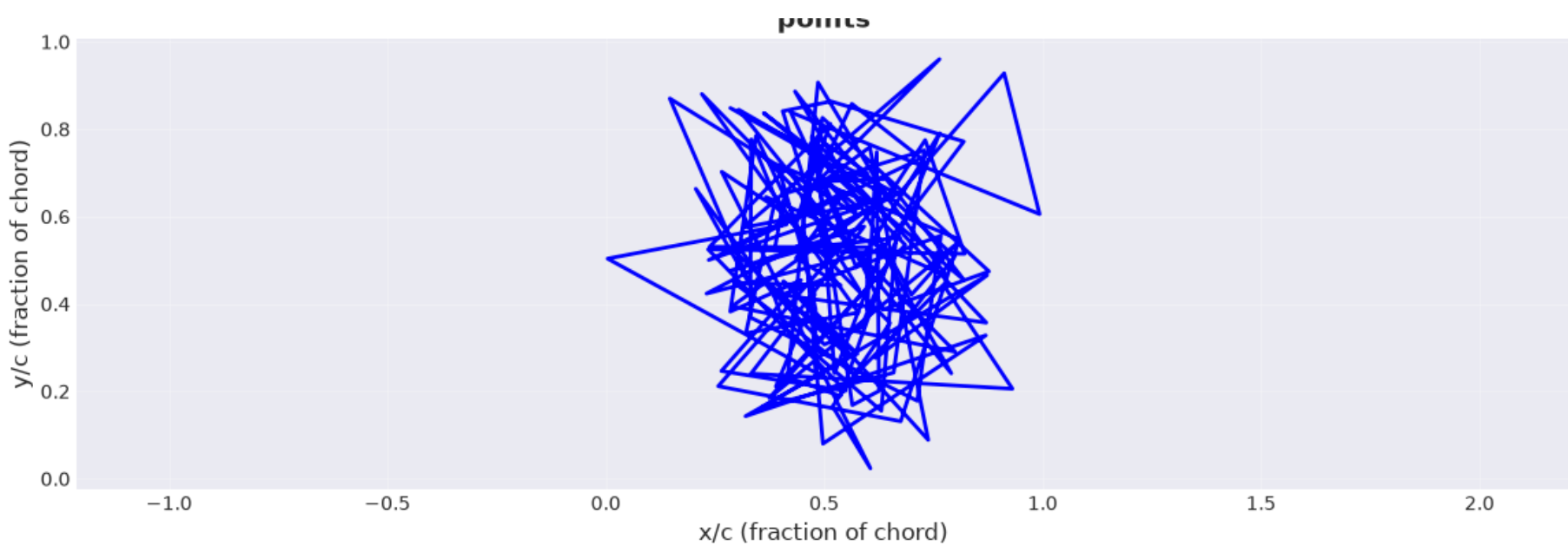
Genetic Algorithm: A type of algorithm that approximates a solution to a complex problem. They act like evolution, where the better the potential solution, the more likely it is to influence the next generation.

CFD: Computation Fluid Dynamics – Using complex computer programs to simulate the flow of fluids around and through a system.

Parameters

Population	10,000
Mutation rate	5%
Elites	50
Generations	1750
Selection	Top half
Fitness	Lift over drag (L/D)

Study 1



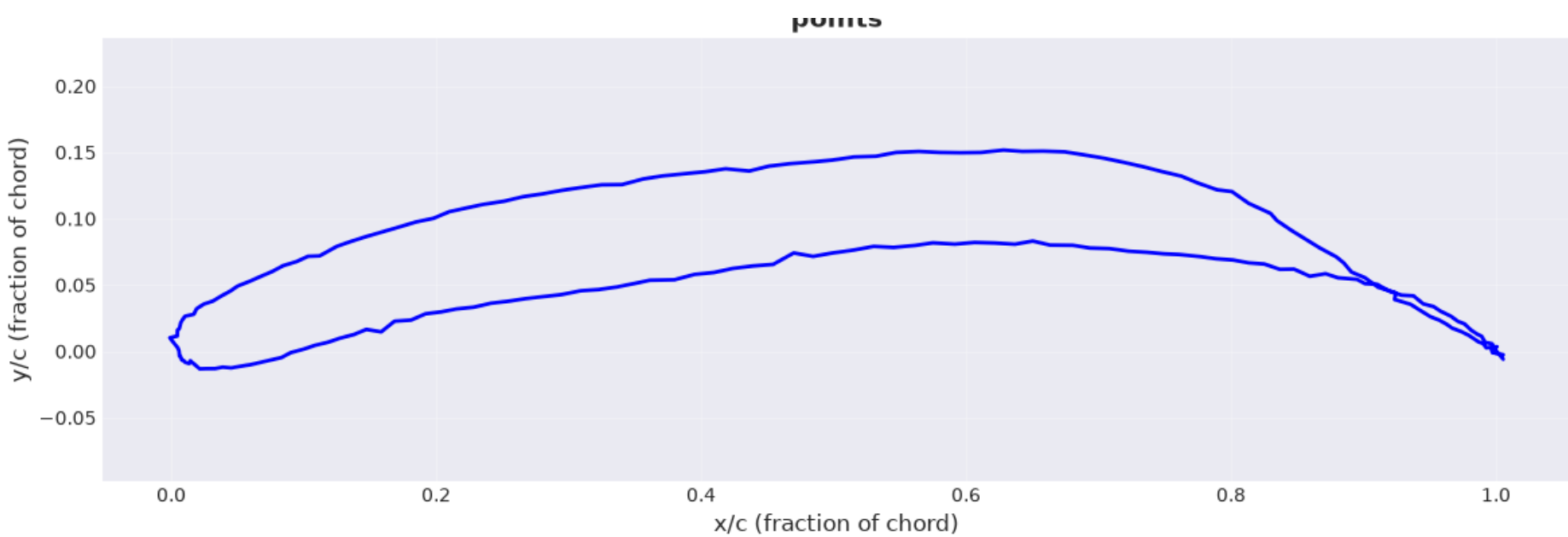
In my first test, I started with a population of random points, and combined the individual points, usually with a bit of mutation. It clearly hasn't worked.

Starting	Random points
Combining	Averaging points
L/D	Infinity



More details ↑

Study 2



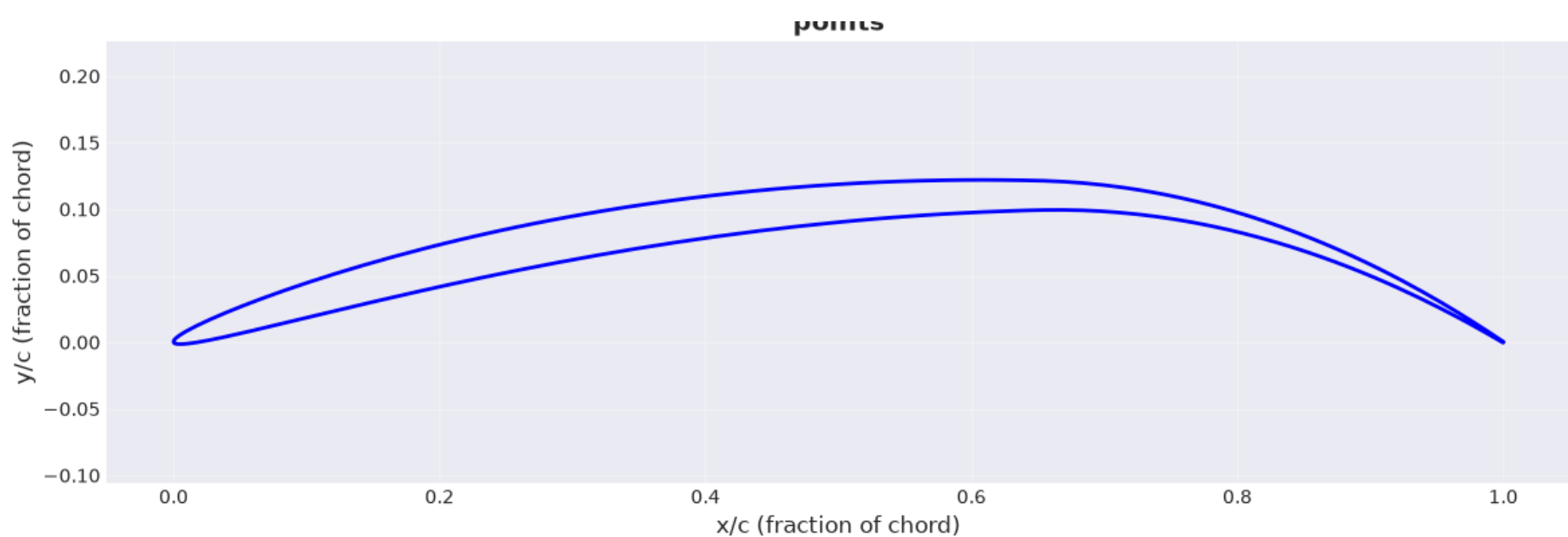
In my second test, I started with a population of NACA 4-digit airfoils, then combined them in the same way as test 1. This worked a lot better, but there is still a bit of bumpiness because the solver isn't perfect.



Starting	Airfoils
Combining	Averaging points
L/D	494

↑ More details

Study 3



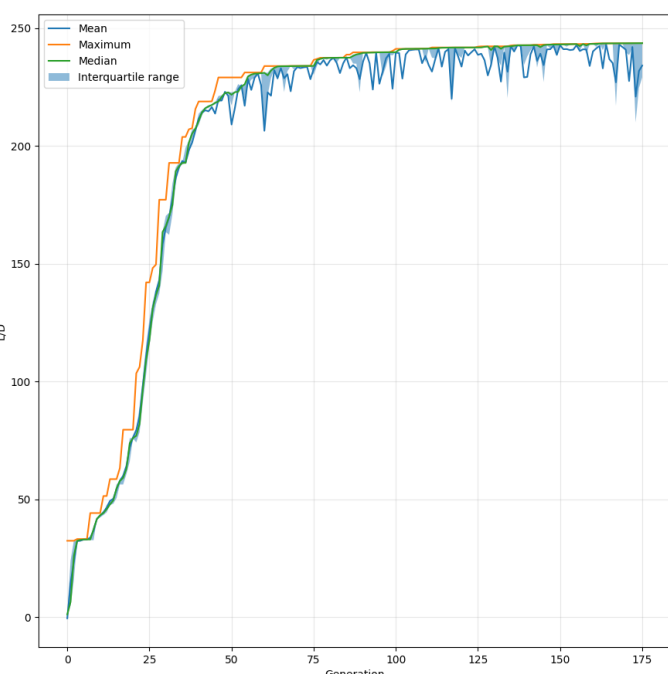
In my third test, I started with NACA 4-digit airfoils, but combined them by averaging the maximum camber deviation, location and thickness of the airfoils.

Starting	Airfoils
Combining	Average parameters
L/D	395



More details ↑

Study 4



In this test, I recorded the maximum, mean and minimum scores of all airfoils in the population. I did the same test as **Study 3**, but I plotted how the L/D changed as the generations progressed.



Mean
Maximum
Median
Interquartile range

↑ More details

Conclusion

- Genetic algorithms are useful to discover if there are any bugs in a solver as they *will* find and exploit them if there are any.
- If the system is properly constrained, like **Study 3**, then genetic algorithms can produce a near-perfect result.
- If left to run for long enough then genetic algorithms will stop producing accurate results as the solver won't give a higher result.

Scan to see all the code I used to make this →

